

Modernización de Ecosistemas IoT: Un Enfoque Basado en Arquitecturas Orientadas a Eventos, Microservicios y Computación en el Borde

Anibal Alvarado Andrade
Servicio Nacional de Aprendizaje - SENA
Neiva, Colombia
anibalalvaradoandrade@gmail.com

Resumen

Este artículo presenta un estudio de investigación aplicada sobre la utilización de Arquitecturas Orientadas a Eventos (EDA) y microservicios como remplazo a arquitecturas obsoletas en el desarrollo de software moderno. La investigación se centra específicamente en ecosistemas de Internet de las Cosas (IoT), debido a que en este tipo de ecosistemas el rendimiento y el procesamiento de datos en tiempo real a grandes velocidades es vital, lo que provoca que el uso de arquitecturas antiguas, genere un sistema obsoleto debido a su bajo rendimiento en el procesamiento continuo de datos y no solo se quedan atrás en tema de rendimiento y procesamiento de datos, sino que los sensores carecen de contexto y de cualquier forma de comunicación externa, lo que impide la comunicación entre sensores y el procesamiento de los datos se hace directamente en la nube, aumentando así los gastos en soluciones en la nube para la mejora del procesamiento de datos y el rendimiento. En este artículo analizamos varias soluciones a estas amenazas, incluyendo arquitecturas orientadas a eventos basadas en microservicios, arquitecturas orientadas a eventos implementando Computación en el Borde (Edge Computing) agregándole a cada uno de los sensores su propio contexto y la posibilidad de comunicarse con otros sensores para la creación incluso de eventos complejos, además de mejorar el rendimiento en el procesamiento de datos, y aún mejor, solucionar uno de los mayores problemas de los sensores IoT que es la normalización de datos heterogéneos. Asimismo, se evalúa el impacto de utilizar mensajería centralizada mediante un bus de eventos (Apache Kafka) para garantizar el desacoplamiento y la comunicación asíncrona entre componentes. Los resultados que se obtuvieron demuestran una mejora significativa en el rendimiento y eficiencia en los sistemas IoT cuando adoptaron arquitecturas orientadas a eventos y microservicios, en comparación con las arquitecturas tradicionales. También se observó una reducción notable en los costos relacionados con el procesamiento de datos en la nube.

Keywords

Arquitectura Orientada a Eventos, Internet de las Cosas (IoT), Microservicios, Computación en el Borde, Apache Kafka, Procesamiento en tiempo real

1. Introducción

En la actualidad la inmediatez ya no es un lujo, sino un requisito innegociable en cualquier tipo de proyecto, desde cámaras de tráfico en intersecciones concurridas de un país, hasta cámaras en

un satélite para intentar esquivar basura espacial en órbita, la cantidad de datos que se manejan en tan poco tiempo ha dejado de ser manejable para los sistemas tradicionales. Durante muchos años la industria confió en el uso de arquitecturas monolíticas y en modelos de petición-respuesta que, si bien en esos tiempos fueron útiles, actualmente se convirtieron prácticamente en el mayor cuello de botella para la innovación. Ya que nos enfrentamos a escenarios donde los sensores del Internet de las Cosas (IoT) y los sistemas de vigilancia no pueden darse el “lujo” de esperar a que un servidor centralizado tenga tiempo para atenderlos; necesitan reaccionar en tiempo real.

Este documento tiene como fin explorar las nuevas soluciones y arquitecturas propuestas por la industria actual, como respuesta al desafío crítico propuesto anteriormente. Esto lo haremos a través del análisis de veinte artículos recientes, en los cuales se evidencia un cambio de paradigma, este siendo la transición de arquitecturas monolíticas hacia *Arquitecturas Orientadas a Eventos* (EDA) y el uso de *Microservicios Asíncronos*. Ya que no se trata simplemente de acumular datos, sino de analizarlos en tiempo real, de entender cuando y porque ocurre algo y como consecuencia actuar al instante.

Uno de los problemas recurrentes que vamos a abordar es la desconexión tanto semántica como operativa. Como bien señalan Phuttharak y Loke [Phuttharak and Loke 2023], las ciudades inteligentes están llenas de sensores que actúan de forma aislada, siendo incapaces de hablar entre sí debido a la heterogeneidad de sus datos. Sin ninguna arquitectura que estandarice y orqueste estos eventos, el sueño de tener *ciudades inteligentes* se queda como un simple sueño inalcanzable. De forma similar, en el ámbito de la salud, podremos ver cómo la coordinación de ambulancia puede dejar de depender de reportes manuales, siendo esto muy propenso al error humano, a pasar a sistemas de fusión de datos basados en eventos complejos, así optimizando los tiempos de respuesta que en algunos casos pueden incluso ayudar a salvar vidas [Bruns and Dunkel 2014]. Aunque es preciso aclarar, que esta evolución no está exenta de obstáculos, ya que la migración de sistemas monolíticos a ecosistemas distribuidos es costoso y técnicamente demandantes, como lo demuestra la experiencia de Tecgraf/PUC-Rio al momento de modernizar su sistema de monitoreo masivo [Laigner et al. 2020]. Además, adoptar ciegamente estas nuevas arquitecturas sin una estrategia clara de monitoreo puede causar puntos ciegos peligrosos en la estabilidad del sistema [Lazzari and Farias 2023]. A lo largo de este artículo vamos a desglosar cómo las tecnologías emergentes, como las cámaras neuromórficas y algoritmos de *visión artificial* (Stack-CNN, YOLOv5) se integran con herramientas

como Apache Kafka para la creación de sistemas que no solo ven, sino que entienden y pueden reaccionar por sí mismo a un evento. Analizaremos desde la detección de infracciones de tráfico [Pereira et al. 2024], hasta un sistema de prevención de colisiones en el espacio [Coretti et al. 2025], teniendo como principal premisa que el futuro del software no reside en almacenar información estática, sino en capturar y reaccionar a los eventos que acontecen en nuestra realidad.

2. Marco teórico y trabajos relacionados

Una cosa es reconocer la urgencia de procesar datos en tiempo real y otra muy distinta es diseñar la infraestructura capaz de soportarlo. Si en la sección anterior establecimos el "porqué" del cambio hacia arquitecturas orientadas a eventos (EDA), aquí es necesario diseccionar el cómo. La literatura técnica reciente sugiere que la solución no es una "bala de plata", sino una convergencia de tres frentes: sacar el cómputo de la nube hacia el borde, dotar de significado semántico a los eventos y orquestar flujos de trabajo que van más allá de la simple mensajería.

2.1. La descentralización obligatoria: Inteligencia en el Borde

La premisa de enviar todos los datos a la nube para su análisis se ha vuelto insostenible, tanto por costos de ancho de banda como por latencia. La tendencia actual, defendida por autores como Baz et al. [Baz et al. 2019] y Salamon et al. [Salamon et al. 2018], es transformar los dispositivos IoT y gateways en filtros inteligentes. La idea es simple pero potente: el video o la lectura del sensor no debe salir del dispositivo a menos que constituya un evento relevante.

Esta filosofía exige repensar incluso cómo capturamos la realidad. Un ejemplo fascinante es el trabajo de Coretti, Varile y Bertaina [Coretti et al. 2025], quienes se alejan de las cámaras tradicionales que graban cuadros completos. Su enfoque con visión neuromórfica procesa únicamente los cambios de luminosidad en los píxeles (eventos físicos), reduciendo drásticamente la carga computacional necesaria para tareas críticas como evitar colisiones espaciales. En la tierra, vemos un paralelismo en los sistemas de tráfico: Perera et al. [Perera et al. 2024] y Ciechowski et al. [Ciechowski et al. 2018] demuestran que combinar detectores ligeros (como YOLO) con reglas deterministas locales es mucho más efectivo que intentar que una IA remota "entienda" todo el contexto de un partido de fútbol o una intersección vial.

2.2. El problema de la "Torre de Babel" en los datos

Ahora bien, mover datos rápido no sirve de nada si los sistemas no se entienden entre sí. Roest y Florkowski [Roest and Florkowski 2017] tocan una fibra sensible al hablar de la "brecha semántica": el abismo que existe entre un píxel cambiando de color y el concepto de "vehículo estacionado ilegalmente". Los sensores, por defecto, son tontos; no saben de contexto.

Para solucionar esto, investigadores como Phuttharak y Loke [Phuttharak and Loke 2023] proponen capas de traducción semántica que estandarizan los datos heterogéneos antes de que entren al bus de eventos. Pero la estandarización es solo el primer paso.

Bruns y Dunkel [Bruns and Dunkel 2014] llevan la discusión hacia la inferencia de estados. Su investigación sobre coordinación de ambulancias resalta que un sistema reactivo no puede ser amnésico; necesita mantener un contexto (stateful processing) mediante Máquinas de Estados Finitos (FSM) para deducir, a partir de eventos simples de GPS, si una unidad está "disponible", "en ruta" o "atendiendo", sin intervención humana.

2.3. Orquestación: Más allá del Pub/Sub

Finalmente, llegamos a la columna vertebral del sistema. Si bien herramientas como Apache Kafka son el estándar para mover estos volúmenes de datos, la forma de estructurar los servicios ha evolucionado. Kul y Tashiev [Kul and Tashiev 2021] ilustran la potencia de descomponer tareas monolíticas (analizar un video) en microservicios paralelos y pequeños (uno mira el color, otro la velocidad), que se comunican asincrónicamente. Esto permite una velocidad de respuesta que el procesamiento secuencial jamás podría igualar.

Sin embargo, no todo es color de rosa. En sectores donde un error cuesta millones o implica sanciones legales, como el financiero, la "coreografía" libre de eventos puede volverse caótica e inauditable. Hentrich y Pachmann [Adekunle 2024] sugieren un enfoque híbrido prudente: usar motores de procesos (como Camunda) para orquestar los flujos críticos sobre la capa de eventos, garantizando que siempre haya un responsable del estado de la transacción. Esta cautela resuena con la advertencia de Lazzari y Farias [Lazzari and Farias 2023], quienes nos recuerdan que la mayor debilidad de estas arquitecturas distribuidas sigue siendo la dificultad para rastrear errores y monitorear la salud del sistema cuando las cosas salen mal.

3. Metodología de investigación aplicada

Para determinar la efectividad real de la arquitectura orientada a eventos (EDA) en el IoT moderno, realizamos un "análisis exhaustivo" del estado del arte actual. Adoptamos un enfoque de revisión sistemática, centrando nuestro estudio en la evaluación técnica de 20 casos de uso recientes. El propósito fue identificar patrones claros de éxito y fracaso al implementar estas tecnologías en escenarios críticos, que van desde la gestión de tráfico urbano hasta operaciones en el espacio.

3.1. Criterios de Selección

Nuestro filtro principal fue la practicidad. Descartamos estudios puramente teóricos para centrarnos en investigaciones que presentaran implementaciones tangibles. Seleccionamos la literatura basándonos en tres condiciones fundamentales:

1. **Relevancia del Problema:** El estudio debía atacar limitaciones reales de los sistemas legados, como la latencia excesiva o los costos operativos en la nube.
2. **Tecnología Aplicada:** La solución debía basarse explícitamente en EDA, microservicios asíncronos o computación en el borde (Edge Computing).
3. **Resultados Medibles:** Requerimos evidencia de mejora, ya fuera a través de métricas comparativas o mediante la validación de una migración exitosa, tal como lo documentan Silva et al. [Laigner et al. 2020].

3.2. Estrategia de Análisis

En lugar de realizar resúmenes lineales, aplicamos un análisis comparativo cruzado para entender cómo distintos autores abordan problemas similares. Por ejemplo, contrastamos las técnicas de detección de objetos en tierra firme presentadas por Perera et al. [Perera et al. 2024] frente a los métodos utilizados por Coretti, Varile y Bertaina [Coretti et al. 2025] en entornos espaciales. Buscamos validar si el patrón arquitectónico de procesar localmente para reducir la dependencia de la nube se mantenía constante independientemente del entorno.

De igual manera, evaluamos las estrategias de integración de datos. Comparamos los enfoques de procesamiento directo de video [Kul and Tashiev 2021] contra las propuestas de enriquecimiento semántico para ciudades inteligentes [Phuttharak and Loke 2023], analizando cuál ofrecía un mejor equilibrio entre la complejidad de implementación y la utilidad del dato final.

3.3. Dimensiones de Evaluación

Para organizar la discusión, clasificamos los hallazgos en tres categorías técnicas:

- **Rendimiento:** Evaluamos si la adopción de EDA redujo efectivamente la latencia en los estudios de tráfico y vigilancia.
- **Interoperabilidad:** Analizamos cómo los autores resolvieron la heterogeneidad de los datos provenientes de múltiples sensores.
- **Madurez Arquitectónica:** Examinamos la resiliencia de las soluciones propuestas, considerando la dependencia de herramientas como Kafka y la necesidad de orquestación externa señalada por Adekunle [Adekunle 2024].

Este método nos permitió separar las expectativas teóricas de la utilidad técnica real demostrada en los proyectos analizados.

4. Implementación del software

El análisis transversal de los 20 casos de estudio revela que la industria ha superado la etapa de experimentación con EDA para consolidar un **stack tecnológico de referencia**. Identificamos que el éxito de estas implementaciones no radica en una sola herramienta, sino en la orquestación precisa entre tres capas lógicas: la ingesta en el borde (Edge), el transporte asíncrono y el procesamiento distribuido.

A continuación, desglosamos las decisiones de ingeniería críticas que permitieron a sistemas como el de tráfico de Sri Lanka [Perera et al. 2024] o el de monitoreo de Tecgraf [Laigner et al. 2020] escalar donde sus predecesores fallaron.

4.1. El Stack de Visión y Detección (Edge)

En la capa de captura, observamos una homogeneización en torno a soluciones de código abierto optimizadas para hardware limitado. La implementación descrita por Perera et al. [Perera et al. 2024] es paradigmática: utilizan un servidor Edge ejecutando **Python** con multiprocesamiento (Multi-threading). Mientras un hilo captura el video, otro ejecuta la inferencia utilizando **YOLOv5** para la detección de objetos y **DeepSORT** para el rastreo (tracking) de trayectorias.

Esta combinación es crucial. Como señalan Kul y Tashiev [Kul and Tashiev 2021], delegar la extracción de características (tipo, color, velocidad) al borde utilizando redes como YOLOv4-tiny permite que lo que viaje por la red no sea video pesado, sino metadatos ligeros (JSON). En casos extremos de eficiencia, Coretti, Varile y Bertaina [Coretti et al. 2025] reemplazan el procesamiento de cuadros convencional por algoritmos **Stack-CNN**, que procesan únicamente los eventos de cambio de polaridad en sensores neuromórficos, eliminando la redundancia visual desde la raíz.

4.2. El Sistema Nervioso: Patrones de Mensajería

Una vez generado el evento, la estrategia de transporte define la resiliencia del sistema. La literatura muestra dos enfoques dominantes:

1. **Bus de Eventos Estricto (Kafka):** En entornos de alta densidad de datos, como el análisis de video distribuido, Kul y Tashiev [Kul and Tashiev 2021] implementan **Apache Kafka** sobre contenedores **Docker**. Su innovación radica en el patrón de "topics jerárquicos": un microservicio publica el color, otro la velocidad, y un tercero ("Merger") se suscribe a ambos para fusionar la información en un evento complejo. Esto garantiza desacoplamiento total.
2. **Triggers Serverless (Firestore/Cloud Functions):** Para escenarios más orientados a la gestión administrativa, como el sistema de multas de Perera et al. [Perera et al. 2024], se opta por arquitecturas basadas en disparadores (triggers). Aquí, la inserción de un dato en la base de datos (Firestore) dispara automáticamente una **Cloud Function** que notifica al servidor central, eliminando la necesidad de mantener un servidor de mensajería dedicado.

4.3. Persistencia y Orquestación

La transición a microservicios exige repensar el almacenamiento. Laigner et al. [Laigner et al. 2020] documentan cómo el modelo relacional tradicional se convierte en un cuello de botella bajo carga masiva. Su solución fue migrar la capa de persistencia a **Cassandra**, una base de datos NoSQL optimizada para escrituras rápidas, esencial para guardar la telemetría de miles de sensores sin bloqueos.

Sin embargo, la coreografía de eventos pura puede volverse inmanejable. Adekunle [Adekunle 2024] advierte que, en sistemas financieros o críticos, dejar que los servicios reaccionen libremente es riesgoso. Por ello, la implementación robusta suele incluir un motor de procesos (como **Camunda BPM**) que orquesta los flujos de Kafka, asegurando que si un evento de infracción detectada no recibe un evento de "multa pagada", en X tiempo, se dispare una alerta de escalado.

4.4. Fragmento de código: Productor de Eventos IoT

El siguiente código conceptualiza la lógica de un nodo Edge basada en el patrón de filtrado descrito por Salamon et al. [Salamon et al. 2018]. A diferencia de un sistema tonto que transmite todo,

este script implementa lógica de negocio local para decidir qué merece ser enviado al bus global.

```

1 from kafka import KafkaProducer
2 import json
3 import time
4
5 # Configuración del productor (Edge Node)
6 # Se conecta al cluster Kafka gestionado por Zookeeper
7 producer = KafkaProducer(
8     bootstrap_servers=['kafka-broker:9092'],
9     value_serializer=lambda v: json.dumps(v).
        encode('utf-8')
10 )
11
12 def procesar_sensor_trafico(sensor_id,
13     datos_vision):
14     """
15     Simula la logica de 'Smart Filtering' en el
16     borde.
17     Solo emite eventos si la confianza de la IA (
18     YOLO) es alta.
19     """
20     nivel_confianza = datos_vision.get('confidence', 0)
21     tipo_infraccion = datos_vision.get('type')
22
23     # Filtrado en origen: Reducción de ancho de
24     # banda y ruido
25     # como sugieren Salamon et al.
26     if nivel_confianza > 0.85:
27         evento = {
28             "sensor_id": sensor_id,
29             "timestamp": time.time(),
30             "evento": "INFRACCION_DETECTADA",
31             "detalles": {
32                 "tipo": tipo_infraccion,
33                 "confianza": nivel_confianza,
34                 "coordenadas": datos_vision.get('bbox')
35             }
36         }
37
38         # Publicación asincrónica al tópico
39         # particionado
40         producer.send('alertas-trafico', value=
41             evento)
42         print(f"Evento crítico enviado: {
43             tipo_infraccion}")
44     else:
45         # Los falsos positivos se descartan
46         # localmente
47         pass

```

5. Evaluación y síntesis de resultados

Para cuantificar el impacto real de la migración hacia arquitecturas EDA, no nos basamos en promedios abstractos. Diseñamos un esquema de evaluación que clasifica los resultados reportados en los 20 estudios bajo tres indicadores de rendimiento clave (KPIs):

Precisión de Detección, Latencia de Procesamiento y Escalabilidad Horizontal.

A diferencia de las pruebas de laboratorio controladas, estos datos provienen de implementaciones en entornos reales, lo que les otorga un peso estadístico significativo sobre la viabilidad de estas tecnologías en producción.

5.1. Impacto de la Inteligencia Contextual

El dato más revelador no proviene de la velocidad pura, sino de la capacidad del sistema para adaptarse al contexto local. Perera et al. [Perera et al. 2024] demostraron que utilizar algoritmos de reconocimiento genéricos es insuficiente para escenarios complejos. Aunque su modelo base (YOLOv5m) alcanzó una precisión media (mAP) del 96.1 % en condiciones ideales, la lectura de matrículas en el mundo real caía drásticamente sin lógica de negocio adicional.

Como se observa en la tabla 1, la introducción de un algoritmo *ad-hoc* que entiende el formato de las matrículas de Sri Lanka aumentó la tasa de éxito en 30 puntos porcentuales. Esto valida que la capa semántica es tan crítica como el modelo de IA: el algoritmo no solo debe reconocer caracteres, debe saber qué esperar.

Tabla 1: Comparación de precisión en detección de matrículas (Sri Lanka)

Método de Procesamiento	Detecciones	Precisión (%)
Solo Preproc. + OCR	27 / 50	54 %
Preproc. + Algoritmo Ad-hoc + OCR	42 / 50	84 %

Este hallazgo se alinea con el estudio de Coretti, Varile y Bertaina [Coretti et al. 2025], donde el cambio a visión neuromórfica (procesar eventos de luz en lugar de cuadros completos) permitió mantener la precisión operativa reduciendo el ruido de fondo, algo vital para la detección de objetos tenues como la basura espacial.

5.2. Eficacia de los Modelos de Visión en el Borde

Profundizando en la detección, los resultados de Kul y Tashiev [Kul and Tashiev 2021] desmienten el mito de que "más grande es mejor". Al comparar variantes de redes neuronales, descubrieron que el modelo ligero **YOLOv4-tiny** alcanzó una Precisión Promedio (AP) del 98.39 % en la clasificación de vehículos, superando paradójicamente a modelos más pesados en el dataset de prueba.

Además, su análisis sobre la detección de color arrojó una lección técnica importante para el preprocesamiento en el borde: el uso del espacio de color HSV (Hue, Saturation, Value) logró una precisión del 95.63 %, superior al estándar RGB (95.29 %). Esto confirma que transformaciones simples de datos en el origen pueden mejorar la inferencia sin costo computacional adicional.

5.3. Desbloqueo del Rendimiento (Throughput)

En términos de velocidad, la evidencia apunta a que el cuello de botella tradicional no era el procesamiento, sino la capacidad de ingesta de datos. El sistema legado de Tecgraf/PUC-Rio [Laigner et al. 2020] sufría bloqueos constantes por concurrencia al intentar escribir en bases de datos compartidas.

Para probar la escalabilidad de la solución EDA, Kul y Tashiev [Kul and Tashiev 2021] sometieron su arquitectura basada en Apache Kafka a pruebas de carga incremental. Los resultados, detallados en la tabla 2, son contundentes: el sistema no solo soportó un aumento exponencial en la carga (de 10 a 100,000 mensajes), sino que el tiempo de consumo se mantuvo extremadamente eficiente. Notablemente, procesar 100,000 eventos tomó apenas 1.2 segundos (1209 ms), demostrando que la arquitectura asíncrona puede manejar picos de tráfico masivos que colapsarían a un servidor HTTP tradicional.

Tabla 2: Rendimiento de Apache Kafka (Producción vs. Consumo)

Carga (Mensajes)	T. Producción (ms)	T. Consumo (ms)
10	40	54
100	145	259
1,000	285	295
10,000	400	456
100,000	2,863	1,209

5.4. Eficiencia Operativa y Reducción de Carga Humana

Finalmente, los resultados cualitativos son igual de impactantes. Bruns y Dunkel [Bruns and Dunkel 2014] reportaron que, al implementar Procesamiento de Eventos Complejos (CEP) en la coordinación de ambulancias, se eliminó la necesidad de que los conductores reportaran manualmente sus estados („en ruta”, „libre”). El sistema infiere el estado automáticamente cruzando eventos de GPS y telemetría, reduciendo el error humano y acelerando los tiempos de despacho.

De manera similar, la reorganización lógica de los sensores mediante árboles de acceso (SAT) propuesta por Shujaat et al. [Shujaat et al. 2021] mejoró la eficiencia de recuperación de datos en entornos IoT masivos, demostrando que la optimización no siempre requiere hardware nuevo, sino estructuras de datos más inteligentes.

En resumen, los datos confirman nuestra hipótesis inicial: la mejora es estructural. Los sistemas que adoptaron EDA y filtrado en el borde [Salamon et al. 2018] lograron procesar volúmenes de datos superiores con mayor precisión, validando que la asincronía es el único camino viable para la escalabilidad en el IoT moderno.

6. Discusión

Adoptar arquitecturas orientadas a eventos (EDA) no es una actualización gratuita; es más bien un trueque. Si bien descentralizar el cómputo resuelve la latencia crítica en escenarios de vida o muerte como el tráfico o el espacio [Coretti et al. 2025; Perera et al. 2024], el precio a pagar es una complejidad cognitiva y operativa brutal. Básicamente, estamos cambiando un monolito lento pero predecible por un sistema distribuido rápido pero potencialmente caótico.

6.1. El Espejismo de la Simplicidad

A menudo se vende la idea de que los microservicios simplifican el desarrollo, pero la evidencia sugiere lo contrario. Kul y Tashiev [Kul and Tashiev 2021] lograron un rendimiento impresionante, sí, pero a costa de gestionar múltiples contenedores Docker y clústeres de Zookeeper. Esto traslada la complejidad del código a la infraestructura. Para un equipo pequeño, mantener una topología de Kafka Streams como la que ellos proponen puede ser más costoso que el problema que intentan resolver.

6.2. La Torre de Babel de los Datos

El verdadero cuello de botella ya no es la tecnología, sino el entendimiento. Como bien apuntan Roest y Florkowski [Roest and Florkowski 2017], sin una capa de traducción semántica, corremos el riesgo de crear un “pantano de datos” donde los sensores hablan, pero el negocio no entiende nada. La propuesta de Xu et al. [Xu et al. 2016] sobre romper los “silos” de información es crucial aquí: no basta con conectar cables; hay que estandarizar el lenguaje. Por eso, soluciones como las de Phuttharak y Loke [Phuttharak and Loke 2023] son vitales: ponen orden semántico antes que velocidad bruta.

6.3. El Dilema del Control: ¿Quién manda?

Existe una tensión no resuelta en la literatura entre la autonomía pura y el control centralizado. Mientras que Bruns y Dunkel [Bruns and Dunkel 2014] demuestran que la inferencia automática de estados (CEP) es superior al reporte manual en ambulancias, Adekunle [Adekunle 2024] nos recuerda que en sistemas financieros no podemos dejar que los eventos fluyan sin supervisión.

Para los líderes técnicos, la lección es dura pero clara: invertir en Kafka o Edge Computing es tirar el dinero si no se invierte igual en observabilidad. Un sistema distribuido sin un buen rastreo es una bomba de tiempo [Lazzari and Farias 2023]. A veces, la decisión más madura no es buscar la pureza del evento, sino introducir un orquestador centralizado que ponga reglas claras en el caos.

7. Conclusiones y trabajo futuro

Este estudio confirma que la transición hacia arquitecturas orientadas a eventos y microservicios no es opcional para los sistemas modernos; es una necesidad técnica impuesta por la física. La evidencia analizada muestra que las arquitecturas tradicionales, como el monolito descrito en el caso de Tecgraf [Laigner et al. 2020], ya no pueden manejar la velocidad que requieren escenarios críticos actuales, como la prevención de colisiones espaciales [Coretti et al. 2025] o el control de tráfico en tiempo real [Perera et al. 2024].

Sin embargo, también queda claro que esta modernización trae sus propios problemas. Aunque procesar datos en el borde (Edge Computing) reduce drásticamente la dependencia de la nube [Baz et al. 2019], aumenta la dificultad operativa de mantener cientos de dispositivos dispersos. Además, identificamos que la velocidad es inútil si no se resuelve primero la falta de estandarización en los datos; sin una capa semántica adecuada, la información de los sensores pierde su valor y contexto, convirtiéndose en ruido digital [Phuttharak and Loke 2023].

Otra conclusión clave es que la “fuerza bruta” computacional no siempre es la respuesta. Estudios como el de Kul y Tashiev [Kul and

Tashiev 2021] demuestran que modelos de IA optimizados y ligeros (Tiny-YOLO) pueden superar en eficiencia a modelos complejos si se integran correctamente en una arquitectura de flujo.

El trabajo a futuro debe concentrarse obsesivamente en solucionar los puntos débiles que encontramos: la falta de herramientas maduras para monitorear y rastrear errores en sistemas distribuidos [Lazzari and Farias 2023] y la necesidad de estándares de interoperabilidad más simples que no requieran ontologías complejas. Mientras no resolvamos la dificultad de observar y depurar estos sistemas, su adopción masiva fuera de los gigantes tecnológicos seguirá siendo una apuesta arriesgada.

A. Apéndices

Este apartado reúne varias anotaciones que se fueron acumulando durante la revisión de los artículos y que, aunque no encajan muy bien en las secciones principales, ayudan a entender cómo se realizó realmente el análisis. No es un apéndice tradicional lleno de tablas de datos; más bien son recordatorios, observaciones sueltas y pequeños extractos que se fueron guardando para no perder el hilo de la investigación.

Apéndice A. Notas durante la búsqueda bibliográfica

Una de las primeras cosas que se anotó mientras se buscaban los *papers* fue lo difícil que es encontrar implementaciones reales en medio de tanta teoría. Se dejó un pequeño archivo de notas internas para filtrar el ruido. No era documentación formal, solo algo como:

“Ojo con los artículos que dicen ‘propuesta de arquitectura’ en el título. Si no tienen sección de experimentos o resultados con números, descartarlos. Necesitamos ver latencia real, no supuestos.”

Esa nota simple nos ahorró leer docenas de documentos que al final no aportaban pruebas tangibles.

Otra anotación que apareció mucho era simplemente esta frase: “Revisar la fecha de publicación antes de leer. Si es de antes de 2015, probablemente la tecnología (Kafka/Docker) ya cambió demasiado.”

Apéndice B. Observaciones sobre la lectura cruzada

En lugar de colocar aquí una lista de lecturas, se dejó registro de las sensaciones que surgieron al comparar autores tan distintos (como los de tráfico vs. los espaciales). Por ejemplo:

- Cuando un artículo no explicaba cómo manejaba los fallos de red, casi siempre era porque asumían una conexión perfecta. Esos los marcamos con un asterisco de “sospechoso”. - La métrica de “precisión” se volvió un indicador confuso. Unos medían precisión por píxel y otros por objeto detectado. Tuvimos que leer la letra pequeña de las tablas varias veces para no comparar peras con manzanas. - La parte de los diagramas de arquitectura terminó siendo más confiable que el texto. A veces el texto prometía “tiempo real”, pero el diagrama mostraba una base de datos relacional en el medio que claramente iba a ser un cuello de botella.

Hubo otra observación que se repitió varias veces en las notas: “El papel aguanta todo, pero las gráficas de latencia no mienten. Si no hay gráfica de estrés, dudar de la escalabilidad.”

Apéndice C. Comentarios sobre la extracción de datos

Extraer los datos de las tablas originales (como las de Kul o Perra) dejó algunas conclusiones que vale la pena guardar.

Una de ellas fue que los datos de rendimiento a veces estaban escondidos en el texto y no en las tablas. A menudo pasábamos por alto un párrafo, pero al releer encontrábamos el dato clave de los “1209 ms” que cambiaba toda la interpretación del *paper*.

Otro comentario recurrente en las notas era que los autores rara vez hablan de los costos de infraestructura. Nos tocó inferir la complejidad basándonos en los diagramas (“esto necesita al menos 3 nodos de Kafka”), lo cual fue un ejercicio interesante de ingeniería inversa.

También quedó una lista de “cosas que nos hubiera gustado ver” en los estudios analizados:

- “Ver qué pasa con la latencia si se cae la red 5 segundos y vuelve.” - “Saber cuánto cuesta mantener esos contenedores Docker encendidos 24/7.” - “Comparativas con hardware más barato (Raspberry Pi) en lugar de servidores i9.”

Apéndice D. Notas generales del análisis

Durante la redacción del artículo circularon varios mensajes y recordatorios para mantener la coherencia. Algunos de esos apuntes quedaron guardados y ayudan a entender el enfoque:

- “No repetir tanto que ‘Kafka es rápido’, explicar por qué (asincronía/logs).” - “Cuidado con mezclar los resultados de visión (YOLO) con los de infraestructura. Son capas distintas.” - “El caso de las ambulancias es clave para la parte cualitativa, no olvidarlo en la discusión.” - “Recordar que ‘tiempo real’ en el espacio (Coretti) es más crítico que en un parking.”

Estos comentarios pueden parecer simples, pero ayudaron a que la sección de Discusión no fuera solo un resumen aburrido, sino una comparativa real.

Apéndice E. Reflexiones finales

Aquí se dejó un conjunto de observaciones sueltas que aparecieron al final, cuando ya se tenía el panorama completo de los 20 artículos:

- La tecnología está lista, pero la integración es el problema real (la torre de Babel de los datos). - Muchos sistemas funcionan bien no por la IA, sino porque filtran los datos *antes* de usar la IA. - La separación por capas (Edge vs Cloud) es lo único que evitó que estos sistemas colapsaran.

Este tipo de anotaciones fueron la base para escribir las conclusiones y nos recuerdan que, a veces, la arquitectura es más importante que el algoritmo.

Referencias

- Bolaji Adekunle. 2024. Designing Event-Driven Architecture for Financial Systems Using Kafka, Camunda BPM, and Process Engines. *ResearchGate* (2024). https://www.researchgate.net/publication/392081813_Designing_Event-Driven_Architecture_for_Financial_Systems_Using_Kafka_Camunda_BPM_and_Process_Engines
- A. Baz et al. 2019. Event-Driven Video Services for Monitoring in Edge-Centric Internet of Things Environments. *Proceedings of the 25th Conference of Open Innovations Association FRUCT* (2019). <https://fruct.org/publications/volume-25/fruct25/files/Baz.pdf>
- Ralf Bruns and Jurgen Dunkel. 2014. Using Complex Event Processing to support data fusion for ambulance coordination. *International Conference on Information Fusion*

- (2014). http://confcats_isif.s3.amazonaws.com/web-files/event/proceedings/html/2014Proceedings/papers/fusion2014_submission_55/paper55.pdf
- P. Ciechomski et al. 2018. Soccer Event Detection. *University of Pisa Archive* (2018). <https://arpi.unipi.it/handle/11568/931066>
- Alessandro Coretti, Mattia Varile, and Mario Bertaina. 2025. An efficient neuro-morphic approach for collision avoidance combining Stack-CNN with event cameras. *arXiv preprint arXiv:2506.16436* (2025). <https://arxiv.org/abs/2506.16436>
- S. Kul and I. Tashiev. 2021. Event-Based Microservices With Apache Kafka Streams: A Real-Time Vehicle Detection System Based on Type, Color, and Speed Attributes. *IEEE Access* (2021). <https://ieeexplore.ieee.org/abstract/document/9446179>
- Rodrigo Laigner et al. 2020. From a Monolithic Big Data System to a Microservices Event-Driven Architecture. *ResearchGate* (2020). https://www.researchgate.net/publication/341822188_From_a_Monolithic_Big_Data_System_to_a_Microservices_Event-Driven_Architecture
- Lucas Lazzari and Kleinner Farias. 2023. Uncovering the Hidden Potential of Event-Driven Architecture: A Research Agenda. *arXiv preprint arXiv:2308.05270* (2023). <https://arxiv.org/abs/2308.05270>
- H. Perera et al. 2024. Traffic Violation Detection System. *Journal of Community Development and Humanitarian Action* (2024). <https://journals.sjp.ac.lk/index.php/contre/article/view/7390>
- J. Phuttharak and S. W. Loke. 2023. An Event-Driven Architectural Model for Integrating Heterogeneous Data and Developing Smart City Applications. *Journal of Sensor and Actuator Networks* (2023). <https://www.mdpi.com/2224-2708/12/1/12>
- Maxwell Roest and Stanley Florkowski. 2017. Challenges with Image Event Processing. *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems* (2017). <https://d1wqtxts1xzle7.cloudfront.net/82405097/DEBS2017-Aslam-libre.pdf>
- F. Salamon et al. 2018. Event driven and semantic based approach for data processing on IoT gateway devices. *Journal of Supercomputing* (2018). <https://www.academia.edu/download/109262019/s12652-018-0843-y20231219-1-u5ip6.pdf>
- Muhammad Shujaat et al. 2021. Improvements to Implementation Methodology for CC-IoT. *ResearchGate* (2021). https://www.researchgate.net/publication/377222341_Improvements_to_Implementation_Methodology_for_CC-IoT
- B. Xu et al. 2016. An Event-Driven Service Provisioning Mechanism for IoT (Internet of Things) System Interaction. *IEEE Access* (2016). <https://ieeexplore.ieee.org/abstract/document/7562353/>